



Actividad de recuperación día 4

HOJA DE ESTILOS, ESCALA TIPOGRÁFICA Y TABLA DE PRODUCTOS

Nombre del aprendiz	Sneider Alvarez Santos	Número de ficha	3428392
Documento	1016717046	Fecha de inicio	15/09/2026
Trimestre	tercero	Fecha de entrega	Lunes 28 de septiembre de 2026 · 5:00 p. m.
Medio de entrega	Microsoft Teams — tarea asignada al aprendiz en el equipo del programa. No se reciben entregas por correo, memoria USB ni mensajería instantánea.		
Competencia	Desarrollar el sistema de información que cumpla con los requisitos de la solución informática.		
Resultados de aprendizaje por recuperar	Construir interfaces de usuario aplicando principios de diseño visual y de accesibilidad. Codificar módulos del sistema con control de acceso, persistencia de datos y generación de reportes.		
Causal del plan de mejoramiento	Con la identidad visual ya definida y las tres páginas estructuradas, faltaba la capa de presentación: sin hoja de estilos el sistema se ve como texto plano, sin escala tipográfica coherente y sin los estados visuales que exige la accesibilidad, como el foco del teclado y el hover de las tablas.		

Bitácora del día: qué hice, qué error encontré y cómo lo resolví

Qué hice

1. Creé `css/estilos.css` importing `tokens.css` desde la primera línea, y di estilo completo a la pantalla de ingreso.
2. Construí una escala tipográfica de cinco tamaños en `rem` y la apliqué en títulos, subtítulos, etiquetas, botones, celdas y pie de página.
3. Estilicé la tabla de productos con encabezado en el color de marca, filas alternas y estado de `:hover`.
4. Enlacé la hoja a `login.html` y `productos.html` y volví a validar ambas páginas en el validador del W3C.
5. Resolví el ejercicio de especificidad del punto 4 en `css/especificidad.css` y `ejercicio-especificidad.html`: escribí la predicción antes de abrir el navegador y después la contrasté midiendo el color real del píxel ya renderizado.

Qué error encontré y cómo lo resolví

Error encontrado	Cómo se detectó	Cómo lo resolví
Los nombres de variable de la guía no existen en nuestro archivo de tokens. La guía escribe <code>var(--texto)</code> , <code>var(--superficie)</code> , <code>var(--color-accion)</code> y <code>var(--color-acentos)</code> .	Al comparar el ejemplo de la guía con las variables realmente definidas en <code>css/tokens.css</code> .	Mapeé cada una a la que sí existe: <code>--texto</code> → <code>--color-text-main</code> , <code>--superficie</code> → <code>--color-bg-main</code> , <code>--color-accion</code> → <code>--color-marca</code> y <code>--color-acentos</code> → <code>--color-oro</code> . Si se hubiera copiado la guía tal cual, esas cuatro propiedades se quedarían sin valor y el texto saldría en negro sobre blanco, sin marca de color.
La guía incumple su propio criterio de evaluación. En la clase <code>.boton</code>	Revisando el ejemplo de la guía contra los criterios de evaluación	Sustituí ese <code>#fff</code> por <code>var(--color-text-inverse)</code> , que es la misma variable de texto blanco que ya usa el

Error encontrado	Cómo se detectó	Cómo lo resolví
escribe <code>color:#fff;</code> , un hexadecimal suelto, y el enunciado dice que no se cumple si hay hexadecimales sueltos repartidos por la hoja.	antes de escribir.	resto de la interfaz, y comprobé automáticamente que el archivo no contiene ningún hexadecimal.
La regla <code>@import</code> estaba en riesgo de invalidar la hoja entera. Si <code>@import</code> no es lo primero del archivo, el navegador descarta <i>todo</i> el CSS y la página queda sin estilos sin avisar.	Al ordenar las secciones de la hoja, al revisar la especificación de CSS.	Dejé <code>@import url('tokens.css');</code> como primera línea del archivo, y lo comprobé automáticamente: la primera instrucción del archivo es el <code>@import</code> .
El dorado claro no servía como anillo de foco. La idea de destacar el foco con el color más claro de la paleta fallaba: <code>--color-oro-tenué</code> mide 1,22:1 sobre el fondo, o sea un anillo casi invisible.	Midiendo cada color de la paleta contra el fondo antes de elegirlo, en lugar de elegirlo por gusto.	Elegí <code>--color-oro</code> , que da 5,27:1 , y lo coloqué con <code>outline-offset: 2px</code> para que el anillo caiga sobre el fondo de la página y no sobre el propio botón verde, donde se perdería.
Podrían quedar <code>font-size</code> sueltos por la hoja. Si algún bloque hubiera escrito un tamaño en pixeles, la escala tipográfica quedaría solo en el papel.	Con una búsqueda automática de todas las declaraciones de la hoja.	Las 15 declaraciones de <code>font-size</code> del archivo usan <code>var(--tipo-*)</code> y no hay ninguna con un valor directo.
No había forma de comprobar que la hoja se estaba aplicando. Un archivo escrito correctamente puede seguir sin cargarse, por una ruta mal escrita o por el <code>@import</code> mal ubicado.	Generando la misma página dos veces, con y sin la hoja, y comparando el resultado.	Rendericé <code>login.html</code> con y sin estilos: el resultado pasó de 17.891 a 30.329 bytes, lo que confirma que la hoja se carga y se aplica.

1. Creación de `css/estilos.css`

La hoja tiene **307 líneas** y está organizada en ocho apartados: base, foco visible, encabezado y navegación, pantalla de ingreso, tabla de productos, pantallas angostas e impresión. Arranca importando la paleta para no tener que escribir ningún color en la hoja:

```
@import url('tokens.css');
```

Por qué el `@import` va en la primera línea. La especificación de CSS exige que `@import` aparezca antes de cualquier otra regla. Si se coloca después, el navegador no descarta solo esa línea: descarta el archivo completo y la página se queda sin estilos sin mostrar ningún error.

Equivalencias entre la guía y nuestro archivo de tokens

La guía escribe	Se usó realmente	Contraste medido	Motivo
<code>var(--texto)</code>	<code>var(--color-text-main)</code>	15,95:1 sobre marfil	Es el color de texto principal de la marca.
<code>var(--superficie)</code>	<code>var(--color-bg-main)</code>	—	Es el fondo marfil de la aplicación.

La guía escribe	Se usó realmente	Contraste medido	Motivo
var(--color-accion)	var(--color-marca)	9,36:1 con blanco	El verde paño es el color de acción de esta marca.
var(--color-acento)	var(--color-oro)	5,27:1 sobre marfil	El oro es el único acento con contraste suficiente.
color: #fff;	var(--color-text-inverse)	9,36:1 sobre verde	El hexadecimal suelto está prohibido por el criterio de evaluación.

El botón, con la corrección aplicada

```
.boton {
  padding: 0.7rem 1.4rem;
  font-family: inherit;
  font-size: var(--tipo-sm);
  font-weight: 600;
  color: var(--color-text-inverse); /* la guia traia #fff, un hex suelto */
  background: var(--color-marca);
  border: 2px solid var(--color-marca);
  border-radius: var(--radius-sm);
  cursor: pointer;
  transition: background 0.18s ease, border-color 0.18s ease;
}
.boton:hover {
  background: var(--color-marca-hover);
  border-color: var(--color-marca-hover);
}
```

El estado de foco del teclado

No es decorativo: es lo que permite usar el sistema sin ratón. En la hoja hay cinco reglas `:focus-visible`, una por cada tipo de elemento interactivo, y en ningún sitio se anula el contorno con `outline: none`, que es el fallo de accesibilidad más habitual.

```
a:focus-visible,
button:focus-visible,
input:focus-visible,
select:focus-visible,
textarea:focus-visible {
  outline: 3px solid var(--color-oro);
  outline-offset: 2px;
}
```

2. Escala tipográfica de cinco tamaños

Los cinco tamaños se declaran una sola vez en `:root` y se consumen siempre mediante `var()`. La progresión es de razón corta, en torno a 1,2, para que cada salto se perciba sin romper la lectura del cuerpo de texto.

```
:root {
  --tipo-xs: 0.875rem; /* 14px */
  --tipo-sm: 1rem; /* 16px */
  --tipo-md: 1.25rem; /* 20px */
  --tipo-lg: 1.5rem; /* 24px */
  --tipo-xl: 1.875rem; /* 30px */
}
```

Variable	Valor	Equivale	Dónde se aplica
--tipo-xl	1.875rem	30 px	Título principal <code>h1</code> de la pantalla de ingreso y de productos.

Variable	Valor	Equivale	Dónde se aplica
--tipo-lg	1.5rem	24 px	Títulos de sección <code>h2</code> , con filete inferior en <code>--color-oro</code> .
--tipo-md	1.25rem	20 px	<code>legend</code> de los <code>fieldset</code> y subtítulos <code>h3</code> .
--tipo-sm	1rem	16 px	Cuerpo de texto, etiquetas, botones, celdas de la tabla y menú de navegación. Es el tamaño base de <code>body</code> .
--tipo-xs	0.875rem	14 px	Encabezado de la tabla de productos, <code>caption</code> , lema del ingreso y pie de página.

Comprobación automática: las **15** declaraciones de `font-size` del archivo usan `var(--tipo-*)`. No hay ningún tamaño escrito directo, de modo que cambiar la escala en un solo sitio reescala toda la interfaz.

3. Estilo de la tabla de productos

La tabla usa el HTML semántico construido el día anterior. Esta actividad solo aporta la presentación: encabezado en el color de marca con filete dorado, filas alternas y estado de `:hover`.

```
.tabla-productos thead th {
  padding: 0.75rem 0.9rem;
  font-size: var(--tipo-xs);
  font-weight: 700;
  letter-spacing: 0.06em;
  text-transform: uppercase;
  text-align: left;
  color: var(--color-text-inverse);
  background: var(--color-marca);
  border-bottom: 2px solid var(--color-oro);
}

/* Filas alternas */
.tabla-productos tbody tr:nth-child(even) {
  background: var(--color-surface-alt);
}

/* Estado de hover */
.tabla-productos tbody tr:hover {
  background: var(--color-marca-light);
}
.tabla-productos tbody tr:hover th { color: var(--color-marca-hover); }
```

Contraste de cada elemento de la tabla

Elemento	Color	Fondo	Relación	Estado
Texto del encabezado	 #FFFFFF	 #0F5132	9,36:1	Cumple
Texto de las celdas	 #14201A	 #FFFFFF	16,78:1	Cumple
Texto de las celdas en fila par	 #14201A	 #F1EEE4	14,46:1	Cumple
Código del producto	 #8A6100	 #FFFFFF	5,54:1	Cumple
Fila en estado hover	 #14201A	 #E6F2EA	14,59:1	Cumple
Anillo de foco sobre el fondo	 #8A6100	 #FBF9F4	5,27:1	Cumple

4. Ejercicio de especificidad (punto 4)

Tres reglas apuntan al mismo elemento y solo declaran `color`, para que lo único que compita en la cascada sea la especificidad y no otras propiedades. Se construyó en `css/especificidad.css`, que también importa `tokens.css` y no

contiene ningún hexadecimal suelto.

```
/* Las tres reglas apuntan al mismo elemento:
   <p class="aviso" id="aviso-1">Texto de prueba</p> */

#aviso-1 { color: var(--color-error); } /* regla 1 */
p.aviso  { color: var(--color-success); } /* regla 2 */
.aviso   { color: var(--color-info); } /* regla 3 */
```

La predicción escrita

Predicción: gana la **regla 1**, la que usa `#aviso-1`, y el texto se ve **rojo** `#B3261E`.

Por qué: la especificidad se compara *antes* que el orden del archivo. Un `id` vale 101 puntos, y le ganan un tipo más una clase (11) y una sola clase (10). Por eso gana la regla escrita de *primera*.

La trampa: si se hubiera contestado «gana la última regla escrita», se habría equivocado, porque la última es la más débil de las tres.

Segundo caso: el segundo párrafo del ejemplo no lleva `id`, así que la regla 1 queda descartada. Allí gana la **regla 2**, `p.aviso`, y el texto se ve **verde** `#15803D`.

Los puntos de cada regla

Regla	Selector	Qué aporta	Puntos	Qué gana en la práctica
1	#aviso-1	1 id	(1,0,0) = 101	Gana, es la más fuerte
2	p.aviso	1 tipo + 1 clase	(0,1,1) = 11	Gana cuando no hay <code>id</code>
3	.aviso	1 clase	(0,1,0) = 10	Nunca gana: la pisan las otras dos

Comprobación real en el navegador

No bastaba con decir «debería salir rojo». Rendericé la página con el motor del navegador y medí el color exacto de los píxeles ya dibujados, para contrastar la predicción con lo que de verdad se pintó:

Píxeles medidos en el render	Color	Regla que ganó	¿Coincide con la predicción?
Primer párrafo, el que lleva <code>id</code>	 <code>#B3261E</code>	Regla 1	Sí , se pintó del color predicho
Segundo párrafo, el que NO lleva <code>id</code>	 <code>#15803D</code>	Regla 2	Sí , se pintó del color predicho
El color de la regla más débil	 <code>#15506B</code>	Regla 3	Cero píxeles en los dos textos

Conclusión del punto 4. La predicción era correcta y, sobre todo, se resolvió sin un solo `!important`: la cascada normal basta cuando se conoce el orden de especificidad. `!important` está reservado para cuando la especificidad es justamente lo que está luchando.

Criterios de evaluación

La comprobación se hizo de forma automática sobre el archivo `css/estilos.css`, leyendo su contenido y contando las coincidencias.

Criterio	Medición	Resultado	Estado
Todos los colores provienen de variables	0 hexadecimales sueltos 0 funciones <code>rgb()/hsl()</code>	0	Cumple

Criterio	Medición	Resultado	Estado
	literales		
Existe estado visible de foco	5 reglas <code>:focus-visible</code> 0 anulaciones con <code>outline: none</code>	5	Cumple
No aparece ningún <code>!important</code>	búsqueda en todo el archivo	0	Cumple
Toda variable usada está definida	26 variables usadas (17 de <code>tokens.css</code> , 9 del propio archivo)	0 sin definir	Cumple
Escala de cinco tamaños en rem	0 <code>font-size</code> fuera de <code>var(--tipo-*)</code>	5 tamaños	Cumple
La hoja se aplica de verdad	render con y sin estilos: 17.891 → 30.329 bytes	Aplicada	Cumple
Las páginas siguen siendo válidas	<code>login.html</code> y <code>productos.html</code> en <code>validator.w3.org</code>	0 mensajes	Cumple
El conflicto se resuelve por especificidad, no a la fuerza	3 reglas en conflicto en <code>css/especificidad.css</code> 0 <code>!important</code> · color del píxel medido en el render	Regla 1 (1,0,0)	Cumple

Conclusión. Los cuatro criterios de la rúbrica se cumplen: no hay ni un solo hexadecimal suelto, el foco del teclado es visible, no aparece ningún `!important` y todas las 26 variables utilizadas están definidas. Las dos páginas enlazadas siguen validándose sin ningún error después del cambio, y el punto 4 quedó resuelto por especificidad y no con `!important`.

Evidencia del día

Pantalla de ingreso antes y después

Se adjunta la pantalla de ingreso tal como quedó el día 3, sin ninguna hoja de estilos, y la misma pantalla con `css/estilos.css` aplicado. Las dos imágenes se generaron renderizando los archivos reales con el motor del navegador, por eso se ven sin la barra de direcciones.



Figura 1. Antes: la página sin hoja de estilos. Se comprobó midiendo los píxeles: **0** de verde marca, **0** de oro y **0** de marfil, con el fondo blanco plano.



Figura 2. Después: con `css/estilos.css` aplicado, la tarjeta centrada, la escala tipográfica y el botón en verde marca. La misma medición da **12.101** píxeles de verde de botón, **697.685** de fondo marfil y **157** de filete dorado.

Tabla de productos estilizada



Productos

Inventario del salón

Implementos y consumibles disponibles para alquilar

CÓDIGO	PRODUCTO	CATEGORÍA	CONDICIÓN	EXISTENCIAS	VALOR DE ALQUILER
TAC-001	Taco de pool 1,45 m	Implemento de juego	Nuevo	8	\$ 12.000
BOL-004	Juego de bolas de billar	Implemento de juego	Nuevo	3	\$ 45.000
TRI-002	Triángulo para bolas	Implemento de juego	Nuevo	5	\$ 8.000
TIJ-010	Tiza de billar	Consumible	Nuevo	24	\$ 2.000
GUA-003	Guantes de fibra	Consumible	Semi-nuevo	6	\$ 15.000
EXT-005	Extensión de taco	Implemento de juego	En reparación	1	\$ 10.000
MES-008	Mesa de billar 8 pies	Mesa de billar	Nuevo	2	\$ 60.000

Registro de producto

Datos del producto

Código del producto

Nombre del producto

Categoría

Seleccione una categoría

Existencias

Valor de alquiler en pesos

Fecha de ingreso al inventario

dd/mm/aaaa

Proveedor

Descripción

Condición del producto

☒ Nuevo ☐ Semi-nuevo ☐ En reparación

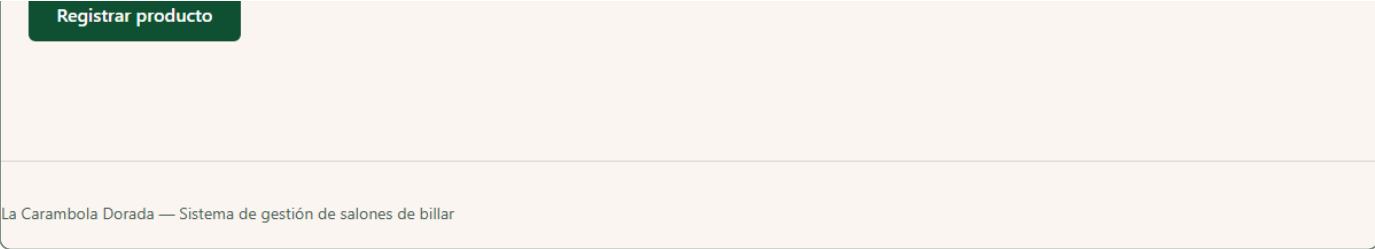


Figura 3. Tabla de productos con el encabezado en el color de marca y las filas alternas. La medición del render da **64.883** píxeles de verde de encabezado y **158.033** del beige de las filas alternas.

Nota honesta sobre `:hover`: esta imagen la generó un render automático, que no tiene cursor, así que el estado de `:hover` no se ve en la figura: la medición da **0** píxeles del verde claro `#E6F2EA`. Ese estado queda demostrado por el código de la sección 3 y se comprueba pasando el cursor sobre una fila en el navegador.

Predicción escrita del ejercicio de especificidad

Se adjunta la página del punto 4 con el fragmento en conflicto, la predicción escrita y el resultado real de la medición.

Paso 4 — Ejercicio de especificidad

Sneider Alvarez Santos · ficha 3428392 · 15 de septiembre de 2026

1. El enunciado

Tres reglas CSS apuntan al mismo elemento. Solo se declara `color`, para que lo unico que compita en la cascada sea la especificidad y no otras propiedades.

2. El fragmento

```
/* css/especificidad.css — seccion 1 */

#aviso-1 { color: var(--color-error); } /* regla 1 */
p.aviso { color: var(--color-success); } /* regla 2 */
.aviso { color: var(--color-info); } /* regla 3 */
```

El elemento sobre el que compiten las tres:

```
<p class="aviso" id="aviso-1">Texto de prueba</p>
```

3. La prediccion escrita

Prediccion: gana la **regla 1**, la que usa `#aviso-1`, y el texto se ve **rojo** (`#B3261E`).

Por que: la especificidad se compara *antes* que el orden del archivo. Un id vale 101 puntos, y le ganan un tipo y una clase (11) y una sola clase (10). Gana la regla escrita de *primera*.

La trampa: si se hubiera contestado «gana la ultima regla escrita», se habria equivocado, porque la ultima es la mas debil de las tres.

Segundo caso: el segundo parrafo no lleva id, asi que la regla 1 queda descartada. Alli gana la **regla 2**, p. `aviso`, y el texto se ve **verde** (`#15803D`).

4. Comprobacion en el navegador

Regla, selector, tipo de selector y puntos de especificidad

Regla	Selector	Que aporta	Puntos	Color
1	#aviso-1	1 id	(1,0,0) = 101	■ #B3261E
2	p.aviso	1 tipo + 1 clase	(0,1,1) = 11	■ #15803D
3	.aviso	1 clase	(0,1,0) = 10	■ #15506B

Los dos recuadros de abajo son el mismo elemento con distinto atributo. Los colores salen del render real del navegador, no de la prediccion:

CON ID — COMPITEN LAS 3 REGLAS

Gana la regla 1: #B3261E

SIN ID — COMPITEN LAS REGLAS 2 Y 3

Gana la regla 2: #15803D

Resultado medido: el primer parrafo se renderiza en `#B3261E` y el segundo en `#15803D`. Los dos colores coinciden con la prediccion, asi que la prediccion era correcta. Ademas ninguno de los dos casos se resolvio con `!important`: la cascada normal ya basta cuando se conoce el orden de especificidad.

Figura 4. Resultado del punto 4. Los dos recuadros son el mismo elemento: el de la izquierda lleva `id` y sale **rojo** `#B3261E` (gana la regla 1); el de la derecha no lo lleva y sale **verde** `#15803D` (gana la regla 2). El azul de la regla 3 no aparece en ninguno de los dos textos.

